CIE-1 Question Paper — Academic Review

Data Structures & Algorithms · RV University, Bengaluru

Second Semester · Academic Year 2025–26

Dear Academic Coordinator,

We are writing to respectfully bring to your attention several observations regarding the CIE-1 question paper for the Data Structures & Algorithms course. This document has been prepared in the spirit of academic accuracy and fairness to students, and is intended as a constructive reference for your review.

Of the 20 questions in the paper, 11 are entirely correct and unambiguous. The remaining 9 questions contain issues that range from typographic errors and missing information to terminological inaccuracies and ambiguous answer keys. Each concern is documented below with full technical reasoning. Where appropriate, we have also suggested the action we respectfully request.

We appreciate your time and consideration, and remain available to discuss any of these points at your convenience.

Total Questions	No Issues Found	Concerns Raised
20	11	9

Category of Concern	Question(s)
Wrong option text	**Q1**
Missing answer	**Q3**
Missing input	**Q8**
Wrong terminology	**Q5**
Ambiguous statement	**Q2, Q6**
Two valid answers	**Q13**
Compiler/standard-dependent	**Q11**
Option labeling typo	**Q17**

A Note on Intent

The purpose of this CIE-1 contention is solely to enable students to learn better — to understand where the assessment may have been unclear, and to ensure that marks accurately reflect a student's understanding of the subject. This document is not intended to challenge the authority of the faculty or the institution in any way. We have the highest respect for our instructors and the academic process, and we raise these points in the spirit of constructive dialogue and continuous improvement. We sincerely hope this review is received in the same spirit.

Detailed Concerns — Questions Requiring Review

Q1	SLL — Odd-Index Print	Incorrect Answer Option

Input: 11->22->33->44->55->66

```
void prints(struct Node* head) {
struct Node* current = head; int index = 1;
while (current != NULL) {
if (index % 2 == 1) { printf("%d ", current->data); }
current = current->next; index++;
}
}
```

A. 11->22->55->66

B. 11->22->33->55

C. 11->33->44->55

D. 11->33->55->66

Paper Answer	D
Correct Answer	**None of the listed options**

Explanation of the Issue

The function increments an index variable starting at 1 and prints elements only when index % 2 == 1, i.e., at positions 1, 3, and 5. Tracing through the input list 11->22->33->44->55->66, those positions correspond to values 11, 33, and 55. The value 66 occupies position 6 (even), so the print condition is never satisfied for it. The actual program output is: 11 33 55.

Option D, marked as the correct answer, reads 11->33->55->66 and includes 66. This does not match the actual output. None of the four listed options matches what the program produces.

Q2	DLL — Address Mapping	Ambiguous — Missing Structural Context

If a node X comprises of three addresses as 2070, 4250, 7300. Relate the mentioned addresses as Node Address, Next Node Address and Previous Node Address. Find the next pointer address of X's previous node.

A. 2070

B. 4250

C. 7300

D. 8400

Paper Answer	A
Correct Answer	**A — but only under an unstated assumption**

Explanation of the Issue

The question provides three raw memory addresses (2070, 4250, 7300) and asks the student to map them to the fields of a doubly linked list node. No struct definition, memory layout diagram, or explicit field ordering is given anywhere in the question.

The marked answer A (2070) is reachable only by assuming the field order is (Node Address, Next, Previous). Under the equally valid ordering (Node Address, Previous, Next), the answer would be 4250 instead. Without a stated field ordering, both interpretations are technically valid and the question is therefore ambiguous.

Q3	SLL — Pairwise Swap	Answer Omitted from Question Paper

Called with list 1,2,3,4,5,6,7. What will be the contents after execution?

```
struct node { int value; struct node *next; };
void rearrange(struct node *list) {
struct node *p, *q; int temp;
if (!list || !list->next) return;
p = list; q = list->next;
while (q) {
temp = p->value; p->value = q->value; q->value = temp;
p = q->next; q = p ? p->next : 0;
}
}
```

A. 1,2,3,4,5,6,7

B. 2,1,4,3,6,5,7

C. 1,3,2,5,4,7,6

D. 2,3,4,5,6,7,1

Paper Answer	(blank — not provided in the paper)
Correct Answer	**B: 2,1,4,3,6,5,7**

Explanation of the Issue

The answer field for this question was left entirely blank in the distributed question paper. The rearrange() function swaps the values of adjacent node pairs. Beginning with the list 1,2,3,4,5,6,7, the pairs (1,2), (3,4), and (5,6) are swapped, leaving node 7 unchanged as it has no pair. The resulting list is 2,1,4,3,6,5,7, corresponding to option B. This has been verified by compiling and executing the code under GCC (C11).

Q5	2D Array Address — Row-Major	Incorrect Use of Technical Terminology

Consider array A[20][10]. Assume 4 words per memory cell, base address 100, row-major order, first element A[0][0]. What is the address of A[11][5]?

(a) 560

(b) 460

(c) 570

(d) 575

Paper Answer	(a) 560
Correct Answer	**560 — only if word is treated as byte (non-standard)**

Explanation of the Issue

The question states 4 words per memory cell. In standard computer architecture terminology, a word is defined as 2 bytes (16-bit) or 4 bytes (32-bit) and is never equivalent to 1 byte.

Applying the standard definition: With 1 word = 2 bytes: Address = 100 + (11x10 + 5) x 8 = 1,020 With 1 word = 4 bytes: Address = 100 + (11x10 + 5) x 16 = 1,940

Neither result matches any listed option. The answer of 560 is obtained only by treating word as synonymous with byte, which contradicts the standard definition of the term in computer architecture.

Q6	LL Operation Identification	Ambiguous — Undefined Variable Role

```
P = create(node);
Info(p) = 10;
Next(P) = NULL;
Next(List) = P;
List = P;
```

A. POP operation

B. Removal of a node

C. Appending a node

D. Modifying an existing node

Paper Answer	C
Correct Answer	**C — only if List is the tail pointer (never stated)**

Explanation of the Issue

The variable List is used in the pseudocode but its role is never defined. In linked list conventions, a pointer named List most commonly refers to the head. Under this interpretation, Next(List) = P inserts the new node P after the head, and List = P overwrites the head pointer with P, losing all nodes previously in the list. This does not constitute an append operation.

The answer C (Appending a node) is valid only if List is assumed to be a tail pointer. Since this is not stated, students who treated List as a head pointer would arrive at a different answer through correct reasoning.

Q8	Recursive SLL — Output	Input List Not Specified

```
void fun(struct node* start) {
if (start == NULL) return;
printf("%d ", start->data);
if (start->next != NULL) fun(start->next->next);
printf("%d ", start->data);
}
```

(a) 1 4 6 6 4 1

(b) 1 3 5 1 3 5

(c) 1 2 3 5

(d) 1 3 5 5 3 1

Paper Answer	D: 1 3 5 5 3 1
Correct Answer	**D — but only for the assumed (unstated) input 1->2->3->4->5->6**

Explanation of the Issue

The question presents a recursive function and asks for its output, but provides no input linked list. The output is entirely dependent on the contents and length of the list passed to the function. For reference, Question 1 in the same paper explicitly states its input as 11->22->33->44->55->66.

The marked answer D (1 3 5 5 3 1) is only correct if the input is assumed to be 1->2->3->4->5->6, which is not mentioned anywhere in the question. A student using a different but equally valid input would produce a different output.

Q11	Array Initialisation — Error vs Warning	Compiler / Standard Dependent

Which of the following describes the error?

```
#include <stdio.h>
int main() {
int arr[3] = {1, 2, 3, 4};
printf("%d", arr[0]);
return 0;
}
```

a) The array initialization has too many elements for the declared size.

b) The printf statement incorrectly accesses the array.

c) The array should be declared as int arr[] for proper initialization.

d) There is no error; the code is valid.

Paper Answer	a
Correct Answer	**a — but compiler and standard dependent**

Explanation of the Issue

The compiler and C standard in use were not specified in the question. Under GCC with the default C11 standard, the statement int arr[3] = {1,2,3,4} produces a compiler warning (excess elements in array initialiser) and not a hard error. The program compiles with exit code 0 and executes, printing 1.

Under these conditions, option d (there is no error; the code is valid) is partially defensible because the compilation does succeed. The behaviour is compiler-flag and standard-dependent, and without that context being specified, the question does not have a single unambiguous answer.

Q13	malloc — Pointer Not Passed Back	Two Equally Valid Answer Options

```
void set(int *to) {
to = (int*)malloc(5 * sizeof(int));
}
void solve() {
int *ptr;
set(ptr);
*ptr = 10;
printf("%d", *ptr);
}
```

(a) 10

(b) Garbage value

(c) Not determined

(d) The program may crash

Paper Answer	(d) The program may crash
Correct Answer	**Both (c) and (d) are valid descriptions of undefined behaviour**

Explanation of the Issue

The function set() receives the pointer ptr by value. Inside set(), the local copy of to is assigned a malloc() allocation, but this does not affect the caller's ptr, which remains uninitialised. When solve() dereferences ptr with *ptr = 10, it reads from an uninitialised pointer — this is undefined behaviour (UB) under the C standard.

Both option (c) Not determined and option (d) The program may crash are accurate descriptions of undefined behaviour. Option (c) is the more technically precise characterisation, since UB by definition means the result is indeterminate. A crash is one possible outcome but not the only one. The question therefore has two equally correct answers.

Q17	2D Array Pointer Arithmetic	Duplicate Option Label (Typographic Error)

The output is:

```
int a[4][5] = {{1,2,3,4,5},{6,7,8,9,10},{11,12,13,14,15},{16,17,18,19,20}};
printf("%d\n", *(*(a+**a+2)+3));
```

(a) 9

(b) 19

(b) -9 <-- typographic error: should be (c)

(d) 20

Paper Answer	(b) 19
Correct Answer	**(b) 19 — numeric answer is correct**

Explanation of the Issue

The expression *(*(a + **a + 2) + 3) evaluates as: **a = a[0][0] = 1, so a + 1 + 2 = a + 3 refers to row 3. Adding 3 gives column 3, yielding a[3][3] = 19. The numeric answer of 19 is correct.

However, two options are both labelled (b) — one for the value 19 and one for the value -9. The third option should have been labelled (c). A student who correctly identified 19 could not unambiguously mark their response, as the correct value and an incorrect value share the same label.

Questions Verified as Correct

The following 11 questions were independently verified through code execution, manual tracing, and reference to the C standard. No concerns were identified.

Q	Topic	Answer	Status
Q4	DLL — Delete Node, Find Prev of 78	**(d) 2700**	**Verified Correct**
Q7	Recursion — f(20,1)	**C: 9**	**Verified Correct**
Q9	Recursion — Static Variable f(5)	**(d) 18**	**Verified Correct**
Q10	Recursion — Binary of 173	**D: 10101101**	**Verified Correct**
Q12	free(NULL) Behaviour	**(d) Executes normally**	**Verified Correct**
Q14	Memory Segments — i, j, *k	**C**	**Verified Correct**
Q15	Local Variable Storage	**D: Stack**	**Verified Correct**
Q16	malloc/calloc Return Type	**C: void ***	**Verified Correct**
Q18	Circular LL — False Statement	**C**	**Verified Correct**
Q19	SLL — Move Last to Front	**D**	**Verified Correct**
Q20	struct — Stack vs Heap	**a) on stack**	**Verified Correct**

We thank you sincerely for your time and consideration. This review was conducted with the sole intent of ensuring a fair and accurate assessment for all students. We remain happy to discuss any of the points raised above at a time convenient to you.

End of Review Document · CIE-1 · Data Structures & Algorithms · RV University, Bengaluru